

Teknoloji Karşılaştırması ve ERP Veri Desenleri

Fikir #1: B2B Tahsilat Kokpiti · Tasarım çalışması ek notu · 29 Eylül 2026

1. Tamamen .NET ile tamamen JS/TS karşılaştırması

Tamamen JS/TS: ajan, backend ve frontend hepsi Node/TypeScript

Artılar	Eksiler
Tek dil. Tipler ajan, backend ve frontend arasında ortak tutulur (örn. zod şemaları).	COM tabanlı ERP SDK'larıyla (Logo Objects, NetOpenX) konuşmak kırılgan (winax veya edge-js gibi köprüler gerekir). Salt-okuma MVP'de sorun değil; ERP'ye geri yazmada sorun olur.
Mevcut araçlarınız zaten Node: abap-adt-api bir npm kütüphanesi. MCP'nin en olgun SDK'sı TypeScript. Bu birikimi olduğu gibi kullanırsınız.	Windows Service olarak çalıştırmak ek sarmalayıcı ister (node-windows, NSSM). Doğal değil ama çalışır.
LLM, WhatsApp ve e-posta SDK'ları genellikle önce TS ve Python için çıkar.	SQL Server'a Windows kimlik doğrulamasıyla bağlanmak native modül ister (msnodesqlv8), kurulumda sorun çıkarabilir.
Hızlı prototip, geniş geliştirici havuzu.	Müşteri sunucusuna kurulan ajanın çok sayıda npm bağımlılığı olur. Kurumsal BT güvenlik incelemesinde soru işareti yaratır.
Ajan tek dosya olarak paketlenabilir (Node SEA veya Bun).	Tek dosya paketleme .NET kadar olgun değil. SAP'nin Node RFC kütüphanesinin (node-rfc) bakımının durdurulduğu biliniyor; ayrıca doğrulanmalı.

Tamamen .NET: ajan ve backend .NET, frontend React veya Blazor

Artılar	Eksiler
Windows Service, SQL Server, COM ve SAP NCo desteği doğal. Ajan için en sağlam yol.	Frontend yine TypeScript (React). Blazor seçilirse zengin arayüz ve işe alım tarafı zayıflar. Pratikte yine iki dil.
Ajan ile backend aynı veri modelini paylaşır.	Mevcut abap-adt-api ve MCP birikiminiz doğrudan taşınmaz.
Güçlü tip sistemi, performans, uzun destek süresi, kurumsal kabul.	Bazı AI ve mesajlaşma SDK'ları .NET'te topluluk seviyesinde kalıyor (düz HTTP ile sorun çözülür).

Güncellenen öneri. Ekibiniz Node ve MCP ile zaten çalıştığı için:

- Backend ve frontend TypeScript.** AI, MCP, e-posta ve WhatsApp katmanı burada.
- Ajan çok ince tutulur:** yalnızca oku ve gönder. MVP'de ajan da Node ile yazılabilir, çünkü okuma SQL veya REST ile yapılıyor.
- ERP'ye COM ile geri yazma gerektiği gün sadece o modül .NET ile yazılır.

2. SAP'deki mevcut sürecin diğer ERP'lerdeki karşılığı

SAP'de kullanılan iki desen:

Desen	SAP'de bugün yapılan	Mantığı
İtme (push)	ABAP background job (SM36) → API → AI	ERP'nin içinden zamanlanmış iş veriyi dışarı gönderir.
Çekme (pull)	cron → MCP + abap-adt-api → AI	Dışarıdaki bir süreç ERP'ye bağlanıp veriyi alır.

Yerli ERP'lerde ABAP gibi bir iç programlama dili ve job altyapısı yok. Bu yüzden itme desenini ERP yerine veritabanı veya işletim sistemi seviyesinde kurmak gerekiyor.

ERP	Çekme (cron + MCP'nin karşılığı)	İtme (SAP job'unun karşılığı)
Logo Tiger/Go	Logo REST servisi veya salt-okuma SQL. Cari tablosu LG_001_CLCARD , vade ve ödeme planı LG_001_01_PAYTRANS (001 firma, 01 dönem numarası).	SQL Server Agent job veya Windows Görev Zamanlayıcı ile bir script, API'ye gönderir.
Netsis	NetOpenX REST servisi veya SQL (TBLCASABIT cari, TBLCAHAR cari hareket).	SQL Server Agent job veya Görev Zamanlayıcı.
Nebim V3	Nebim Integrator (REST/JSON) veya SQL (örn. cdCurrAcc cari tabloları).	SQL Server Agent job; Integrator üzerinden tetikleme.
Mikro	Mikro API veya SQL (CARI_HESAPLAR , CARI_HESAP_HAREKETLERI).	SQL Server Agent job veya Görev Zamanlayıcı.

Tablo adları bilinen yapılar ama sürüme göre değişebilir. Her müşteride kurulum sırasında doğrulanmalı.

Önemli: Yerli ERP'lerde ADT'nin karşılığı yok. MCP ile yapılmak istenirse kendi MCP sunucunuzu yazarsınız. Bu sunucu ya ERP'nin REST servisini sarar ya da salt-okuma SQL view'larını açar. Aslında tasarımdaki “bağlantı ajanı” tam olarak bu işi, yani çekme desenini, tüm ERP'ler için standart hale getiriyor.

Diğer ERP'lerde uçtan uca süreç

- Kurulumda müşterinin veritabanında salt-okuma bir kullanıcı ve birkaç view açılır: v_acik_kalem , v_cariler , v_tahsilatlar . ERP'ye özel karmaşıklık bu view'larda kalır.
- Ajan (veya cron ile çalışan bir script) 10-15 dakikada bir yalnızca değişen kayıtları okur ve API'ye gönderir.
- Backend veriyi ortak modele yazar, hatırlatma akışlarını tetikler.
- AI yalnızca bu verinin üzerinde çalışır: gelen yanıtları sınıflandırır, mesaj taslağı yazar.

Tasarım açısından iki uyarı

1. Veri taşıma işi deterministik olmalı, AI o yolda olmamalı. Cron + MCP + AI keşif ve ad-hoc soru sormak için çok iyi. Ama fatura ve bakiye senkronizasyonu LLM üzerinden yapılırsa, bir kaydın atlanması durumunda “ödemiş müşteriye hatırlatma gitti” hatası oluşur. Önerilen ayırım:

- Senkronizasyon:** düz konnektör.
- MCP:** uzmanın “bu müşterinin son 6 aylık ödeme davranışı nasıl?” gibi sorularını sorduğu bir özellik (yol haritası adayı).

2. SAP müşterilerinde ADT'yi üretim veri çekimi için kullanmayın. ADT bir geliştirici arayüzü. Geliştirici yetkisi (S_DEVELOP) ister ve canlı sistemlerde genelde kısıtlıdır. Ayrıca SAP'nin dolaylı kullanım (indirect/digital access) lisanslama konusu var. Ürün için öneriler:

- Zaten bilinen ABAP job ile itme, doğrudan Veri Alım API'sine.
- Ya da CDS view'den yayınlanan bir OData servisi.
- Böylece SAP müşterilerinde ajan kurmaya bile gerek kalmaz.

Açık karar: Backend TypeScript'e mi çevrilsin, yoksa .NET'te mi kalınsın? Karar sonrası mimari dokümanı güncellenecek ve Bölüm 2 (veri modeli ve senkronizasyon) ile devam edilecek.